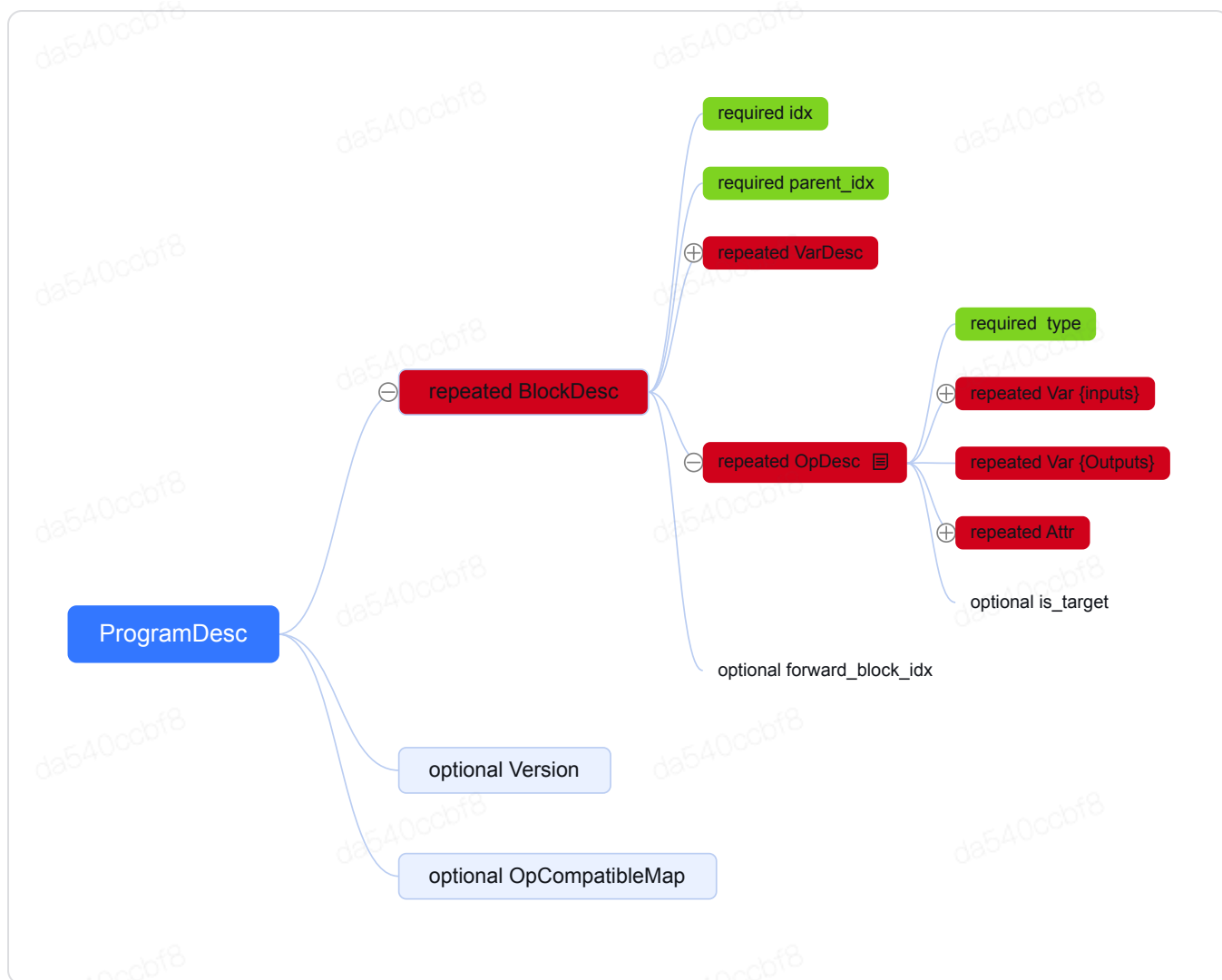


# PaddleLite IR-PASS系统-从模型到RuntimeProgram

## 目录

- **0. Framework.proto**
- **1. Program with Workspace**
- **2. mutable SSA Graph**
- **3. 计算图优化**
  - 4. Runtime Instruction Quene
- **5. 硬件后端抽象**
  - 5.1 kernel接入方式
    - a. 硬件资源抽象
    - b. 子图接入方式
  - 5.2 SSAGraph-->subgraph-->子图接入方式
  - 5.3 执行态算子逻辑

## 0. Framework.proto



## 1. Program with Workspace

protobuf压缩后的模型结构通过proto::Program形式描述了计算图。Lite 根据解析模型后生成的 proto::Program实例对象来准备运行环境（生成必要的变量、创建 OpLite 对象），并生成 cpp::Program实例对象。cpp::Program 是单图多跑 Clone 所要创建的单元，从磁盘中加载一次 proto::Program，可以生成多个 cpp::Program，它们共享权重，独立保存自身的执行态信息。

</> lite/core/program.h

C++ | 收起 ^

```

1 struct Program {
2 public:
3   explicit Program(const std::shared_ptr<Scope>& root) { scope_ = root; }
4   Program(const cpp::ProgramDesc& desc,
5           const std::shared_ptr<Scope>& root,
6           const std::vector<Place>& valid_places)
7       : scope_(root), valid_places_(valid_places), desc_(desc) {
8       PrepareWorkspace(desc);
9       Build(desc);
10  }

```

```
11  std::unique_ptr<Program> Clone() const {
12      std::unique_ptr<Program> res(new Program(desc_, scope_,
13          valid_places_));
14      return res;
15  private:
16      void Build(const cpp::ProgramDesc& program);
17      void PrepareWorkspace(const cpp::ProgramDesc& program);
18  private:
19      std::list<std::string> tmp_vars_;
20      std::list<std::string> weights_;
21      std::list<std::shared_ptr<OpLite>> ops_;
22      std::shared_ptr<lite::Scope> scope_;
23      std::vector<Place> valid_places_;
24      lite::Scope* exec_scope_{};
25      cpp::ProgramDesc desc_;
26  };
```

## 2. mutable SSA Graph

**i** 为了提供统一的辨别执行顺序，切分、融合子图，Lite 在 `cpp::Program` 基础上又抽象了一层 SSA Graph (`mir::graph`)。其特点是为遍历提供了迭代器，并为增删提供接口，向下对接不同的计算图执行环境（单机、分布式、服务器推理、端侧推理、边缘侧推理）的需求。

**i** `mir::Graph` 由 `mir::Node` 组成。`Node` 是一种容器，唯一地对应计算图中的某个节点并保存其前后节点的指针。Paddle 计算图的节点角色仅有两种，`Argument`（数据节点，是一个矩阵，保存计算中间结果或权重）或 `Statement`（一个算子，包含算法单元 `Operator` 和设备上的具体实现 `Kernel`）。

</> Lite/core/optimizer/mir/node.h

C++ | 收起 ^

```
1 // Node in a MIR graph.
2 class Node {
3 public:
4     std::list<Node*> inlinks;
5     std::list<Node*> outlinks;
6
7     Node() = default;
8
9     enum class Role {
10         kArg = 0,
```

```
11     kStmt,  
12     kNumRoles, /*should be last*/  
13     kUnk,  
14 };  
15  
16 class Stmt {  
17     // The kernel instances this Statement contains.  
18     std::vector<std::unique_ptr<KernelBase>> valid_kernels_;  
19     // TODO(Superjomn) make this a shared_ptr for resource safety.  
20     std::shared_ptr<OpLite> op_; // we hold op to run InferShape  
21  
22     public:  
23 };  
24  
25 struct Arg {  
26     std::string name;  
27     int id{0};  
28     const Type* type{nullptr};  
29     // Weight is a special kind of argument, it is marked as weight  
explicitly  
30     // so that some weight related optimization can take place.  
31     bool is_weight{false};  
32     // is_persist indicate that whether is the argument trans from  
Weight  
33     // if the need more than one tool operator(eg. io_copy layout  
calib), the  
34     // argument between them should be persist to make sure it's only  
run once  
35     bool is_persist{false};  
36     int lane{-1};  
37 };  
38  
39 private:  
40     // Either stmt_ or argument_ is used.  
41     std::unique_ptr<Stmt> stmt_;  
42     std::unique_ptr<Arg> arg_;  
43     Role role_{Role::kUnk};  
44     int id_;  
45 };  
46
```

SSA Graph 在 MIR 阶段生成，由 OpLite 组成，包含了 Kernel，以及模型中所有 Variable 类型信息。

### 3. 计算图优化

- ④ 应用在 SSA Graph 数据结构迭代器的算法称为 `mir::Pass`，通过多个 `Pass` 的处理，最终生成可以执行的 `KernelLite` 序列或者其他等价的数据结构，可直接用来执行。

</> lite/core/optimizer/optimize.cc

C++ | 收起 ^

```
1 std::unique_ptr<RuntimeProgram> Optimizer::Run(Program&& program) {
2     auto block_size = program.block_size();
3     for (size_t block_idx = 0; block_idx < block_size; ++block_idx) {
4         std::unique_ptr<mir::SSAGraph> graph;
5         graph.reset(new mir::SSAGraph);
6         graph->Build(program, valid_places_, block_idx);
7         graph->SetValidPlaces(valid_places_);
8         graphs_.emplace_back(std::move(graph));
9     }
10    SpecifyKernelPickTactic(kernel_pick_factor_);
11    InitTargetTypeTransformPass();
12    InitControlFlowOpSharedInputsAndOutputsPlaceSyncPass();
13
14    ApplyPasses(&graphs_);
15
16    exec_scope_ = program.exec_scope();
17
18    return GenRuntimeProgram(&graphs_);
19 }
20
21 // 将经过图优化后的图结构保存成运行时结构
22 std::unique_ptr<RuntimeProgram> Optimizer::GenRuntimeProgram(
23     std::vector<std::unique_ptr<mir::SSAGraph>>* graphs) {
24     auto pass =
25         mir::PassManager::Global().LookUp<mir::GenerateProgramPass>(
26             "generate_program_pass");
27     for (auto& graph : *graphs) {
28         pass->Apply(graph);
29     }
30     auto program = pass->GenProgram();
31     CHECK(exec_scope_);
32     program->set_exec_scope(exec_scope_);
33     return program;
34 }
```

几个重要的 Pass：

- `static_kernel_pick_pass`: 从 graph 中, 挑选合适的 KernelLite 实现;
- `variable_place_inference_pass`: 根据挑选好的 kernel 的 Place 反向推导模型中 Variable 类型。
- `io_complement_pass`: 对需要跨 Target 的 Variable 插入 IoCopy Op, 实现显式拷贝。
- `runtime_context_assign_pass`: 定义每个 KernelLite 的执行上下文, 支持每个 kernel 有不同的 context

## 4. Runtime Instruction Quene

 模型图结构(源码) ---> 计算图表示 (IR) ---> 运行时kernel (目标平台指令)

经过编译期的模型结构 (model+param) 被保存为一个 RuntimeProgram 对象, 其包含一个 Scope对象指针和一个由若干Instruction (运行时kernel) 组成的向量。执行期, RuntimeProgram会依次调用Instruction向量成员 (具体后端kernel) 的Run函数。

</> lite/core/program.h

C++

收起 ^

```
1 struct Instruction {
2     Instruction(const std::shared_ptr<OpLite>& op,
3                 std::unique_ptr<KernelBase>&& kernel)
4         : op_(op), kernel_(std::move(kernel)) {
5         std::string op_type = op->Type();
6         if (op_type == "feed" || op_type == "fetch") {
7             is_feed_fetch_op_ = true;
8         }
9     }
10    // Run the instruction.
11    void Run();
12    private:
13    std::shared_ptr<OpLite> op_;
14    std::unique_ptr<KernelBase> kernel_;
15    bool is_feed_fetch_op_{false};
16    bool first_epoch_{true};
17    bool has_run_{false};
18 };
19
20 class LITE_API RuntimeProgram {
21     public:
22     explicit RuntimeProgram(std::vector<std::vector<Instruction>>&& insts)
23         : instructions_(std::move(insts)) {
24         Init();
25     }
```

```
26  explicit RuntimeProgram(  
27      const std::shared_ptr<const cpp::ProgramDesc>& program_desc,  
28      Scope* exec_scope,  
29      int block_idx = kRootBlockIdx,  
30      bool use_precision_low = false);  
31  bool use_precision_low_ = false;  
32  ~RuntimeProgram() {  
33  }  
34  
35  void Init() {  
36      if (instructions_.empty()) {  
37          LOG(FATAL) << "no instructions";  
38      }  
39      // set context for special target  
40      for (auto& inst : instructions_[kRootBlockIdx]) {  
41          KernelBase* kernel = inst.mutable_kernel();  
42          if (kernel->target() == TARGET(kOpenCL)) {  
43              } else if (kernel->target() == TARGET(kMetal)) {  
44              } else {  
45                  if (kernel != nullptr) {  
46                      kernel->SetContext(  
47                          ContextScheduler::Global().NewContext(kernel->target()));  
48                      }  
49              }  
50      }  
51  }  
52  
53  void Run();  
54  
55  #ifndef LITE_ON_TINY_PUBLISH  
56      // Update the ops and vars of all of blocks to the given program_desc  
57      // according to the instructions  
58      void SaveRuntimeProgramIntoProgramDesc(  
59          std::shared_ptr<cpp::ProgramDesc> program_desc);  
60  #endif  
61  
62  private:  
63      RuntimeProgram(const RuntimeProgram&) = delete;  
64      std::vector<std::vector<Instruction>> instructions_;  
65      Scope* exec_scope_{};  
66      int64_t version_{0};  
67  };
```

## 5. 硬件后端抽象

### 5.1 kernel接入方式

- 对于硬件的通用行为，使用 `TargetWrapper` 模块做适配器适配，对上层框架提供一致的API接口。
- 框架上层策略保持硬件无关，如存储优化 (Memory optimize), 计算剪枝 (Computation prune) 等，任何硬件接入均可直接复用
- 框架支持了硬件通用行为，特定硬件细节不做过多约束，各硬件可以自行实现并接入框架
- 计算模式上目前支持两种主流模型，一种是类似 X86, ARM CPU 等非异构设备；一种是 GPU, 或 FPGA 等异构设备（支持 stream, event异步执行模式以及跨设备拷贝）

#### a. 硬件资源抽象

使用全局单例Devs封装所有的驱动设备，包括计算卡句柄、数据流等数据结构。

</> third-party/opencv/include/CL/CL2.hpp

C++ | 收起 ^

```
1  /*! \brief Class interface for cl_device_id.
2  *
3  * \note Copies of these objects are inexpensive, since they don't
4  *      own
5  *      any underlying resources or data structures.
6  * \see cl_device_id
7  */
8  class Device : public detail::Wrapper<cl_device_id>
9  {
10 public:
11     /*! \brief Default constructor - initializes to NULL.
12     Device() : detail::Wrapper<cl_type>() { }
13
14     /*! \brief Constructor from cl_device_id.
15     *
16     * This simply copies the device ID value, which is an inexpensive
17     * operation.
18     */
19     explicit Device(const cl_device_id &device, bool retainObject =
false) :
        detail::Wrapper<cl_type>(device, retainObject) { }
```



20

&lt;/&gt; lite/core/device\_info.h

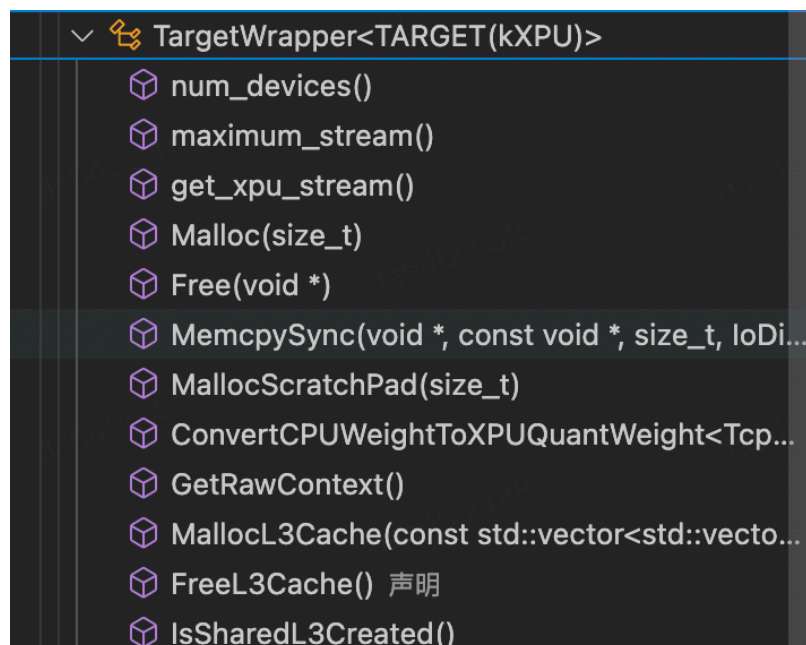
C++ | 收起 ^

```
1 template <TargetType Type>
2 class Device;
3
4 template <TargetType Type>
5 class Env {
6 public:
7     typedef TargetWrapper<Type> API;
8     typedef std::vector<Device<Type>> Devs;
9     static Devs& Global() {
10         static Devs* devs = new Devs();
11         return *devs;
12     }
13     static void Init(int max_stream = 6) {
14     }
15 };
```

&lt;/&gt; lite/backend/xpu/target\_wrapper.h

C++ | 收起 ^

```
1 template <>
2 class TargetWrapper<TARGET(kXPU)> {
3 public:
4     static size_t num_devices() { return 1; }
5     static size_t maximum_stream() { return 0; }
6     ...
7 private:
8     static void ScatterL3Cache(
9         void* l3_ptr,
10        size_t l3_size,
11        const std::vector<std::vector<int64_t>>& query_shape);
12     static void* shared_l3_ptr_;
13     static std::mutex mutex_l3_;
14 };
```



## b. 子图接入方式

若底层驱动设备提供子图方式接入的模型推理引擎，则需要将与计算图有关的数据转换、计算图生成等函数统一抽象为 Lite 接口。

</> lite/backends/xpu/device.h

C++ | 收起 ^

```

1 class Device {
2 public:
3     static Device& Global() {
4         static Device x;
5         return x;
6     }
7     Device() {
8         ...}
9
10    // Build the XPU graph to the XPU runtime, return the XPU runtime
    which can be
11    // used to run inference.
12    std::unique_ptr<xtcl::network::xRuntimeInstance> Build(
13        xtcl::network::xNetworkBuilder* builder,
14        xtcl::network::xTensorCompiler::ParamNDArrayMap* params,
15        std::vector<xtcl::xExpr*>* outputs);
16
17    const std::string name() const { return name_; }
18    const std::string target() const { return target_; }
19
20 private:
21     std::string name_{" "};
22     std::string target_{" "};

```

```
23 };  
24
```

&lt;/&gt; lite/backends/xpu/device.cc

C++ | 收起 ^

```
1 std::unique_ptr<xtcl::network::xRuntimeInstance> Device::Build(  
2     xtcl::network::xNetworkBuilder* builder,  
3     xtcl::network::xTensorCompiler::ParamNDArrayMap* params,  
4     std::vector<xtcl::xExpr*>* outputs) {  
5     VLOG(3) << "[XPU] Build model";  
6     CHECK(builder != nullptr);  
7     CHECK(outputs != nullptr);  
8     CHECK_GT(outputs->size(), 0);  
9  
10    // The XPU compiler build the graph and fill all of the constant  
    params, and  
11    // use TupleNode to support multiple outputs  
12    xtcl::Array<xtcl::xExpr> all_outs;  
13    for (size_t i = 0; i < outputs->size(); i++) {  
14        all_outs.push_back(*outputs->at(i));  
15    }  
16    xtcl::xFunction network =  
17        builder->FinalizeNetwork(xtcl::relay::Tuple(all_outs));  
18    auto target = xtcl::NullValue<xtcl::Target>();  
19    if (!target_.empty()) {  
20        target = xtcl::Target(target_);  
21    }  
22    xtcl::network::xTensorCompiler compiler(network, target);  
23    compiler.SetParams(*params); // Set the data of constant tensors  
24    compiler.Build();  
25    VLOG(3) << "[XPU] Build done";  
26  
27    int device_id = 0;  
28    auto device_str = std::getenv("XPU_VISIBLE_DEVICES");  
29    if (device_str && atoi(device_str)) device_id = atoi(device_str);  
30    VLOG(3) << "[XPU] device id: " << device_id;  
31  
32    return std::unique_ptr<xtcl::network::xRuntimeInstance>(  
33        compiler.CreateRuntimeInstancePtr(device_id));  
34 }
```

## 5.2 SSAGraph-->subgraph-->子图接入方式

❗ 设备相关的加速库因为包含自身融合或其它原因，接受的算子参数形式与框架原生算子定义往往不同，此时需要使用Mutable SSA Graph这一中间形态的数据结构进行转换（满足不同后端【单机、分布式、端侧、边缘侧】对计算图结构的需求）。

❗ 而上述的这些行为是通过一系列的mir::Pass来实现的。它类似于Illum中的IR+PASS，来对不同的src code进行翻译，并最终生成对应目标平台上的执行指令。mir::Pass 为一个纯虚基类，其并不能直接用来创建对象，必须使用其代表派生的某一类型：ProgramPass、StmtPass 和 DebugPass。

</> lite/core/optimizer/mir/pass.h

C++ | 收起 ^

```
1 class Pass {
2 public:
3     // Some appoint here, one pass should be only one of the following
    kinds.
4     enum class Kind {
5         // Will modify the program/graph topology.
6         kProgramWise = 0,
7         // Will modify the statement, with the graph topology fixed.
8         kStmtWise,
9         // Will not modify the IR, just collect information or
        visualization.
10        kDebug,
11    };
12    explicit Pass(Kind kind) : kind_(kind) {}
13
14    virtual void Apply(const std::unique_ptr<SSAGraph>& graph) = 0;
15
16    virtual ~Pass() = default;
17 private:
18     const Kind kind_;
19     std::string name_;
20     std::string doc_;
21     std::set<TargetType> bound_targets_;
22     std::set<TargetType> excluded_targets_;
23     std::map<std::string, std::set<lite_api::Place>> bound_kernels_;
24     std::map<std::string, variant<Node, std::vector<Node*>>> pass_attrs_;
25 };
```

</> lite/core/optimizer/mir/pass.h

C++ | 收起 ^

```
1 class ProgramPass : public Pass {
```

```
2 public:
3   ProgramPass() : Pass(Kind::kProgramWise) {}
4 };
5
6 class StmtPass : public Pass {
7 public:
8   StmtPass() : Pass(Kind::kStmtWise) {}
9 };
10
11 class DebugPass : public Pass {
12 public:
13   DebugPass() : Pass(Kind::kDebug) {}
14 };
```

接下来以用于子图转换的 SubgraphProgramPass为例，对Pass体系进行介绍。为了以子图方式接入不同的后端设备，该ProgramPass的主要功能是根据底层设备支持的算子类型，在 mir::Graph 中找到一部分彼此连接的节点（Nodes）融合转换成底层设备支持的算子形式（Node）。其主要定义如下：

</> lite/core/optimizer/mir/subgraph/subgraph\_pass.h

C++ | 收起 ^

```
1 class NNAdapterSubgraphPass : public ProgramPass {
2 public:
3   void Apply(const std::unique_ptr<SSAGraph>& graph) override;
4 };
```

## 5.3 执行态算子逻辑

**i** 执行态算子即运行在实际设备上的逻辑实现，不同后端设备的实现各不相同。